SeedSmash: Interactive Bot Tournament Platform for Super Smash Bros. Melee

Victor Villin

Short Project Report

SeedSmash is an online platform where user-submitted Reinforcement Learning (RL) agents, called *seeds*, compete in *Super Smash Bros. Melee* (SSBM). The project has two main goals: (1) to provide an accessible and interactive environment where anyone can design, deploy, and observe RL bots, and (2) to serve as a meta-game exploration tool for professional SSBM players. SeedSmash aspires to evolve into a platform for Competitive Reinforcement Learning with Human Feedback (CRLHF), where users compete through their own bots, shaping the agents' learning in real-time via feedback.

This *personal* project, developed alongside my PhD research, uses entirely personal computing resources and has been actively under development since 2022.

Keywords: *user-friendly reinforcement learning, human feedback, multi-agent systems.*

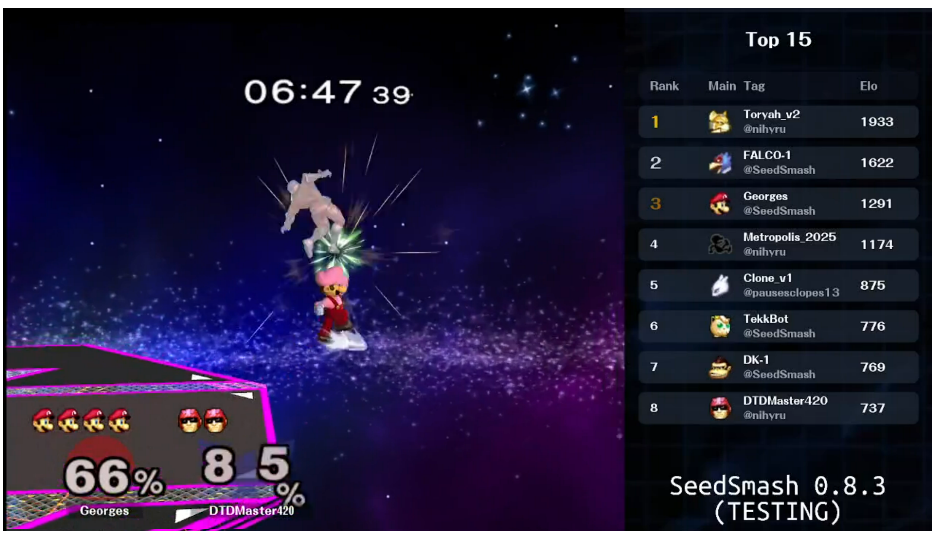

Figure 1: SeedSmash live-stream on twitch.tv.

1 Goal

Super Smash Bros. Melee (SSBM) is a high-paced, technical fighting game released in 2001 for the Nintendo GameCube. It has a thriving professional scene with tournaments drawing hundreds of thousands of viewers. SSBM provides a challenging and rich environment for RL research, requiring agents to generalise across complex states and react with human-like timing [1, 2].

SeedSmash leverages this environment to create a user-centered RL platform where users actively control the learning process of their bots. Early iterations focused on building agents capable of human-level play and diverse behaviours (Daboy, Georges).

© 2025 SeedSmash. All rights reserved.

Objective: Provide an intuitive interface for users to configure RL agents, train them against others, and monitor progress. The interface and training code are available on seedsmash.ai and github.com.

2 User Interface

Users customise their bots through a rich panel (Figure 3): choosing characters, setting strategic preferences, and selecting cosmetic options. Preferences directly modify the underlying *reward function*, guiding agent behaviour (e.g., an aggressive bot prioritises dealing damage over avoiding it). Once submitted, bots enter the SeedSmash database and begin learning by competing against others. Gameplay stats and progression are continuously updated, providing an intuitive overview of performance (Figure 4).

3 Training Bots

Bots learn via proximal policy optimisation [5] using user-configured reward functions. Policies are parameterised with recurrent networks and act on delayed observations (200ms) to emulate human reaction time, encouraging realistic gameplay.

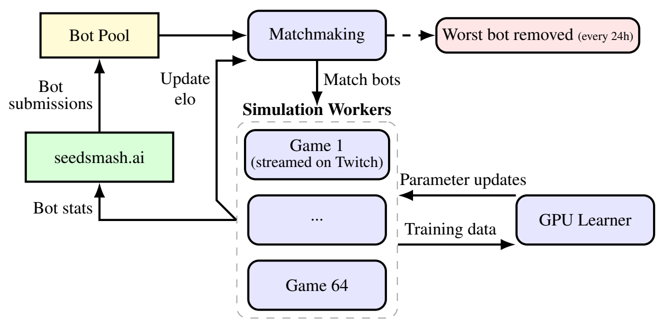

Figure 2: SeedSmash Learning Architecture.

The platform simulates 64 concurrent SSBM instances, generating approximately 3 hours of gameplay per minute. Data generation and policy optimisation are handled through a self-developed multi-agent RL toolkit, Polaris, inspired by RLlib and based on Ray [3, 4]. Figure 2 shows the overall architecture.

4 Preliminary Observations

Early testers describe SeedSmash as both an *intuitive* introduction to RL and an *engaging* gameplay experience.

> ‘As an old time SSBM appreciator, building my own AI and watch it learn in a pond with all their alike is fascinating and full of surprises.’ — *Nihyru, SeedSmash tester*

After a week of continuous training, several bots begin to exhibit human-like gameplay and even discover new strategies. Certain characters, however, tend to dominate due to intrinsic balance differences in the game (see the official tier list). For example, agents playing as Fox learned to perform ‘waveshines’, a powerful and technically demanding technique that is difficult to counter.

5 Future Directions

Currently, user configuration is limited to opaque preferences (e.g., “aggressive” vs. “defensive”), which offers ambiguous control over actual strategies and can result in unintuitive learning dynamics. To address this, we plan to:

1. **Mitigate tier effects:** slow policy updates for top-ranked bots, giving others more time to adapt.
2. **Integrate user feedback:** allow users to guide their bots in real-time via Twitch chat, by indicating if they enjoyed demonstrated gameplay or not. Competing through their own bots, users are naturally incentivised to provide meaningful and strategic feedback, creating a novel human-in-the-loop RL ecosystem: Competitive Reinforcement Learning with Human Feedback (CRLHF).

References

[1] Vlad Firoiu, William F Whitney, and Joshua B Tenenbaum. Beating the world's best at super smash bros. with deep reinforcement learning. *arXiv preprint arXiv:1702.06230*, 2017.

[2] Vlad Firoiu, Tina Ju, and Josh Tenenbaum. At human speed: Deep reinforcement learning with action delay. *arXiv preprint arXiv:1810.07286*, 2018.

[3] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Joseph Gonzalez, Ken Goldberg, and Ion Stoica. Ray rllib: A composable and scalable reinforcement learning library. *arXiv preprint arXiv:1712.09381*, 85:245, 2017.

[4] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, pages 561–577, 2018.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Appendix

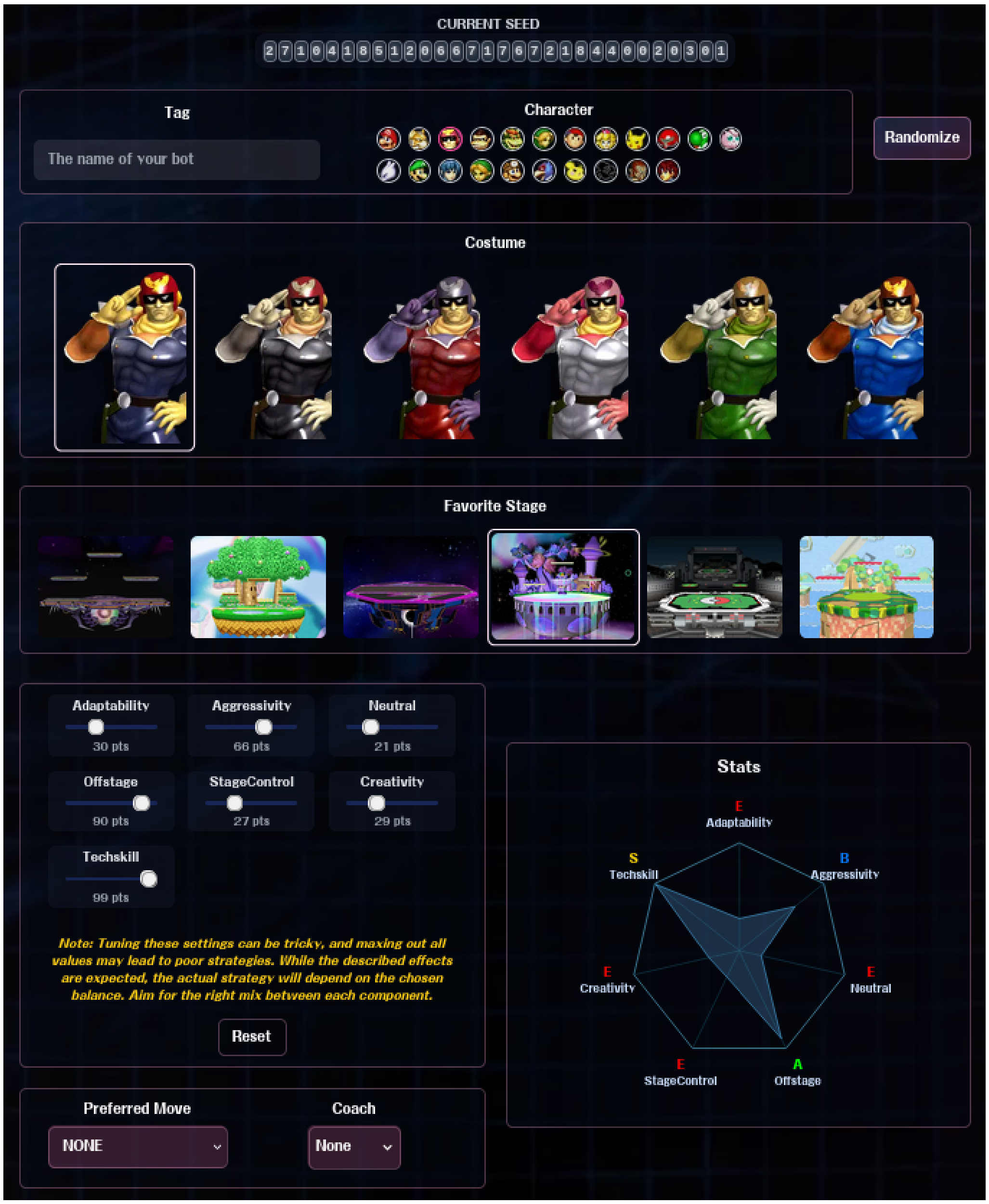

Figure 3: Bot customisation interface.

Figure 4: Bot stat tracking interface.